HIGH-YIELD REVIEW

CSS Exam Cheat Sheet

Built around the COITB HTML-CSS Web Designer objectives. CSS basics, layout, and responsive design account for most of the published blueprint, so those areas receive the most space here.

Exam format	90 items / 90 minutes / single-best-answer and scenario questions
CSS focus	Basics 25% | Layout 20% | Responsive 20% | Advanced 5%
Also tested	HTML structure 20% | Forms 10%
Pacing	About 60 seconds per question; mark uncertain items and return

How to use this: Read the rule, predict the result before revealing the explanation, then type the example from memory. Focus first on the highest-weight domains.

1. Cascade, Selectors, and Specificity

The cascade decides which declaration wins

- First compare origin and importance (user agent, user, author; normal vs `!important`).
- Then compare cascade layer, specificity, scope/proximity, and finally source order.
- Inheritance supplies a value only when no winning declaration sets that property on the element.
- Use `inherit`, `initial`, `unset`, or `revert` deliberately.

Selector part	Weight	Example
Inline style	1-0-0-0	style="..."
ID	0-1-0-0	#menu
Class / attribute / pseudo-class	0-0-1-0	.card, [open], :hover
Type / pseudo-element	0-0-0-1	button, ::before
Universal / combinator	0	*, >, +, ~, space

```
#app .card:hover p { color: navy; } /* 1 ID, 2 class-like, 1 type */
.card p { color: gray; }        /* loses on specificity */
```

Exam trap: Specificity is not a simple point total. Compare each column from left to right. An ID beats any number of classes in the next column.

Selector quick reference

Pattern	Matches
A B	B anywhere inside A
A > B	B that is a direct child of A
A + B	first B immediately after A
A ~ B	later sibling B elements after A
[type='email']	attribute/value match
:first-child	element that is first among siblings
:nth-child(2n)	even-positioned children
:not(.active)	does not match .active
:is(h1, h2)	matches h1 or h2; uses most specific argument
:where(h1, h2)	matches h1 or h2; always zero specificity
::before	generated pseudo-element; needs content

Inheritance

- Commonly inherited: `color`, `font-*`, `line-height`, `text-align`, `visibility`.
- Usually not inherited: dimensions, margin, padding, border, background, position, display.

```
:root { --brand: #175cd3; }
button { color: var(--brand); }
button.special { color: var(--button-color, var(--brand)); }
```

2. Box Model, Sizing, and Flow

Box model

From inside out: content -> padding -> border -> margin. Background paints through content and padding, and under the border by default; margin is transparent.

```
*, *::before, *::after { box-sizing: border-box; }
.card {
  width: 320px;       /* includes padding + border */
  padding: 16px;
  border: 2px solid;
  margin: 0 auto;     /* horizontal centering with width */
}
```

Content-box math: Rendered width = width + left/right padding + left/right border. Horizontal margins affect occupied space but are outside the border box.

Display behavior

Value	Key behavior
block	new line; auto width fills available inline space
inline	flows with text; width/height do not size the box normally
inline-block	flows inline but accepts width/height
none	removed from layout and accessibility tree
flex / grid	outer block box; children use flex/grid layout

Normal flow, overflow, and visibility

- `visibility: hidden` keeps layout space; `display: none` does not.
- `overflow: auto` adds scrolling only when needed; `hidden` clips.
- Vertical margins between normal-flow block elements may collapse. Flex and grid item margins do not collapse.
- Use logical properties such as `margin-inline` and `padding-block` for writing-mode friendly layouts.

Positioning

position	Reference and flow
static	normal flow; offsets do not apply
relative	stays in flow; offsets from its normal position; establishes containing block for abs children
absolute	removed from flow; positioned against nearest positioned ancestor
fixed	removed from flow; usually positioned against viewport
sticky	normal flow until threshold such as top: 0 is reached within scroll container

z-index: It compares stacking levels only within the relevant stacking context. A huge z-index cannot escape an ancestor stacking context.

3. Flexbox - One-Dimensional Layout

Mental model

- Set `display: flex` on the parent. Direct children become flex items.
- Main axis comes from `flex-direction`. `justify-content` aligns on main; `align-items` aligns on cross.
- gap creates space between items without outer-edge margins.

```
.toolbar {
  display: flex;
  flex-flow: row wrap;
  justify-content: space-between; /* main axis */
  align-items: center;            /* cross axis */
  gap: 0.75rem;
}
.toolbar__search { flex: 1 1 18rem; }
.toolbar__button { flex: 0 0 auto; }
```

Property	Meaning
flex-direction	row, row-reverse, column, column-reverse
flex-wrap	nowrap, wrap, wrap-reverse
flex-grow	share positive free space
flex-shrink	shrink when space is insufficient
flex-basis	starting main size before grow/shrink
align-self	one item's cross-axis alignment
align-content	aligns multiple flex lines; needs wrapping and extra cross space
order	visual order only; can confuse keyboard/screen-reader order

Common fix: A flex child may refuse to shrink because its automatic minimum size is content-based. Try `min-width: 0` on that item.

Exam trap: For `flex-direction: column`, the main axis is vertical. Therefore justify-content acts vertically and align-items acts horizontally.

4. CSS Grid - Two-Dimensional Layout

Core patterns

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(14rem, 1fr)
);
  gap: 1rem;
}
.featured { grid-column: span 2; }
```

- Tracks are rows and columns. Grid lines bound tracks; cells are single intersections; areas are rectangular groups of cells.
- `fr` divides available free space. `minmax(min, max)` constrains a track.
- `auto-fit` collapses empty repeated tracks; `auto-fill` preserves their slots.
- Implicit tracks are created when content is placed outside the explicit grid; control them with `grid-auto-rows/columns`.

Placement and alignment

Property	Use
grid-column: 1 / 3	start at line 1, end before line 3
grid-column: span 2	span two column tracks
grid-area: hero	place into named area
justify-items	align items inside cells on inline axis
align-items	align items inside cells on block axis
justify-content	align entire grid when grid is smaller than container
place-items	align-items then justify-items shorthand

```
.page {
  display: grid;
  grid-template-areas:
    'header header'
    'nav    main'
    'footer footer';
  grid-template-columns: 14rem 1fr;
}
main { grid-area: main; min-width: 0; }
```

Flex vs Grid: Choose Flexbox when one axis drives the layout; choose Grid when rows and columns must coordinate. They work well together.

5. Responsive Design

Mobile-first media queries

```
/* Base rules target narrow screens */
.layout { display: grid; gap: 1rem; }

@media (min-width: 48rem) {
  .layout { grid-template-columns: 16rem 1fr; }
}
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto; animation-
duration: 0.01ms; }
}
```

Unit/function	Best use
px	borders and truly fixed small details
%	relative to containing block or property-defined reference
em	relative to current element font size; compounds when nested
rem	relative to root font size; predictable spacing/type
vw / vh	viewport dimensions; beware mobile viewport behavior
dvh / svh / lvh	dynamic, small, and large viewport heights
ch	approximate width of the 0 glyph; useful for readable lines
clamp(min, preferred, max)	fluid value with hard bounds

```
h1 { font-size: clamp(2rem, 5vw, 4.5rem); }
.prose { width: min(100% - 2rem, 65ch); margin-inline: auto;
 }
img { max-width: 100%; height: auto; display: block; }
```

Required HTML partner: Responsive pages normally need `<meta name="viewport" content="width=device-width, initial-scale=1">` in the document head.

Container queries

```
.card-shell { container-type: inline-size; }
@container (min-width: 32rem) {
  .card { grid-template-columns: 10rem 1fr; }
}
```

Use media queries for viewport/environment conditions. Use container queries when a component should respond to its own available space.

6. Typography, Color, Backgrounds, and Effects

Typography

- A robust stack ends with a generic family: `font-family: Inter, Arial, sans-serif`.
- Unitless `line-height` scales with font size and inherits cleanly: `line-height: 1.5`.
- Use `font-weight`, not fake bold; ensure the selected font file supports the weight.
- Keep body text readable, preserve zoom, and avoid setting long paragraphs in all caps.

Color and backgrounds

```
.banner {
  color: #fff;
  background: linear-gradient(120deg, #175cd3, #6938ef);
  box-shadow: 0 10px 30px rgb(16 24 40 / 18%);
}
.avatar { object-fit: cover; aspect-ratio: 1; border-radius:
 50%; }
```

Color form	Example / note
hex	#175CD3; #RGB and #RRGGBB forms
rgb()	rgb(23 92 211 / 80%)
hsl()	hsl(216 81% 46%)
currentColor	uses computed color; useful for borders/icons
transparent	fully transparent color

7. Forms, States, and Accessibility

```
label { display: block; font-weight: 700; }
input { border: 1px solid #98a2b3; padding: .65rem .75rem; }
input:focus-visible { outline: 3px solid #84caff; outline-of
fset: 2px; }
input:invalid:not(:placeholder-shown) { border-color: #d92d2
0; }
button:disabled { opacity: .55; cursor: not-allowed; }
```

- Prefer `:focus-visible` for a visible keyboard focus indicator. Never remove outlines without a strong replacement.
- Do not communicate state by color alone. Pair color with text, an icon, pattern, or other cue.
- Keep visual order consistent with DOM order. CSS reordering does not reliably change reading or tab order.
- Use semantic HTML first. ARIA can describe behavior, but it does not add keyboard behavior automatically.
- `opacity: 0` content can remain interactive; `visibility: hidden` and `display: none` generally remove it from interaction/accessibility.

State selectors

Selector	Typical purpose
:hover	pointer hover; never make it the only access to content
:focus	element has focus
:focus-visible	focus should be visibly indicated
:checked	checked radio/checkbox
:required / :optional	form requirement state
:valid / :invalid	constraint-validation result
:disabled / :enabled	control availability
:has(selector)	select an element based on matching descendants/relative selector

8. Variables, Transitions, Animations, and Sass

Custom properties

```
:root {
  --space-2: .5rem;
  --brand: #175cd3;
}
[data-theme='dark'] { --brand: #84caff; }
.button { padding: var(--space-2) calc(var(--space-2) * 2);
}
```

Custom properties participate in the cascade, inherit by default, and are resolved when used. Their names are case-sensitive.

Transitions and animations

```
.button { transition: transform 160ms ease, background-color
 160ms ease; }
.button:hover { transform: translateY(-2px); }

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: .55; }
}
.status { animation: pulse 1.2s ease-in-out infinite; }
```

- A transition animates between old and new values after a state change. List properties explicitly instead of using `all`.
- An animation uses named `@keyframes`; it can run without a user-triggered property change.
- Transforms and opacity are usually smoother than animating layout properties such as width, left, or top.

Sass / SCSS essentials

```
$brand: #175cd3;
@mixin center { display: flex; align-items: center; justify-
content: center; }
.card {
  @include center;
  &__title { color: $brand; }
  &:hover { transform: translateY(-2px); }
}
```

Exam trap: Browsers do not execute Sass directly. A build step compiles .scss/.sass into standard CSS. CSS custom properties remain available at runtime; Sass variables do not.

9. Shorthands and Common Traps

Clockwise shorthand

Values	Meaning
margin: 8px	all sides
margin: 8px 16px	vertical | horizontal
margin: 8px 16px 24px	top | horizontal | bottom
margin: 8px 16px 24px 32px	top | right | bottom | left

Rapid trap list

- A shorthand can reset omitted longhand values. Example: `background` may reset background-image and related properties.
- Percent heights often need a definite containing-block height; `min-height: 100vh` behaves differently from `height: 100%`.
- `width: 100%` plus padding can overflow under content-box sizing.
- Pseudo-elements need `content` to generate a box in normal use.
- `position: sticky` needs an inset such as `top: 0` and can be constrained by the nearest scrolling ancestor.
- `object-fit` affects replaced content inside its box; `background-size` affects a background image.
- Class selectors are reusable; IDs should be unique in HTML and have high specificity.
- Source order breaks a tie only after earlier cascade criteria tie.

10. Five-Minute Self-Test

1. Which wins: `#app .note` or `body main .note.active`?

Answer: #app .note. The ID column wins before class and type columns are compared.

2. With `flex-direction: column`, which property centers items vertically?

Answer: justify-content, because the main axis is vertical.

3. What does `box-sizing: border-box` change?

Answer: The declared width/height include padding and border.

4. Which removes an element from layout: visibility hidden or display none?

Answer: display: none. visibility: hidden keeps the space.

5. What is the difference between `auto-fit` and `auto-fill`?

Answer: auto-fit collapses empty repeated tracks; auto-fill keeps the empty slots.

6. Does z-index always compare globally?

Answer: No. It is constrained by stacking contexts.

7. Which query style is normally mobile-first?

Answer: Base narrow-screen rules followed by min-width media queries.

8. Can a browser load SCSS directly as CSS?

Answer: No. SCSS must be compiled to CSS.

Sources and Scope

Exam scope and weighting: COITB, HTML & CSS Web Designer Exam Overview, https://www.coitb.org/certifications/html-css-web-designer (accessed 2026-09-30).

Supporting technical reference: MDN Web Docs, CSS Reference and guides, https://developer.mozilla.org/en-US/docs/Web/CSS (accessed 2026-09-30).

This is an independent study aid, not an official COITB publication or a reproduction of live exam questions. The published blueprint can change; verify current details before test day.